ĐỒ ÁN TIN HỌC — HỌC KỲ I (2026–2027)

TÀI LIỆU THIẾT KẾ PHẦN MỀM

Đề tài: Xây dựng chương trình quản lý thư viện bằng ngôn ngữ C++

Giảng viên: Lương An Vinh
Phiên bản tài liệu: 1.0 — Tuần 2

Mục tiêu thiết kế

Chuyển các yêu cầu trong SRS thành thiết kế có thể triển khai: phân chia module, xác định cấu trúc dữ liệu, định nghĩa struct, luồng menu, quy tắc phối hợp giữa các module và kế hoạch lưu file. Thiết kế ưu tiên dễ hiểu, dễ kiểm thử và phù hợp với C++11.

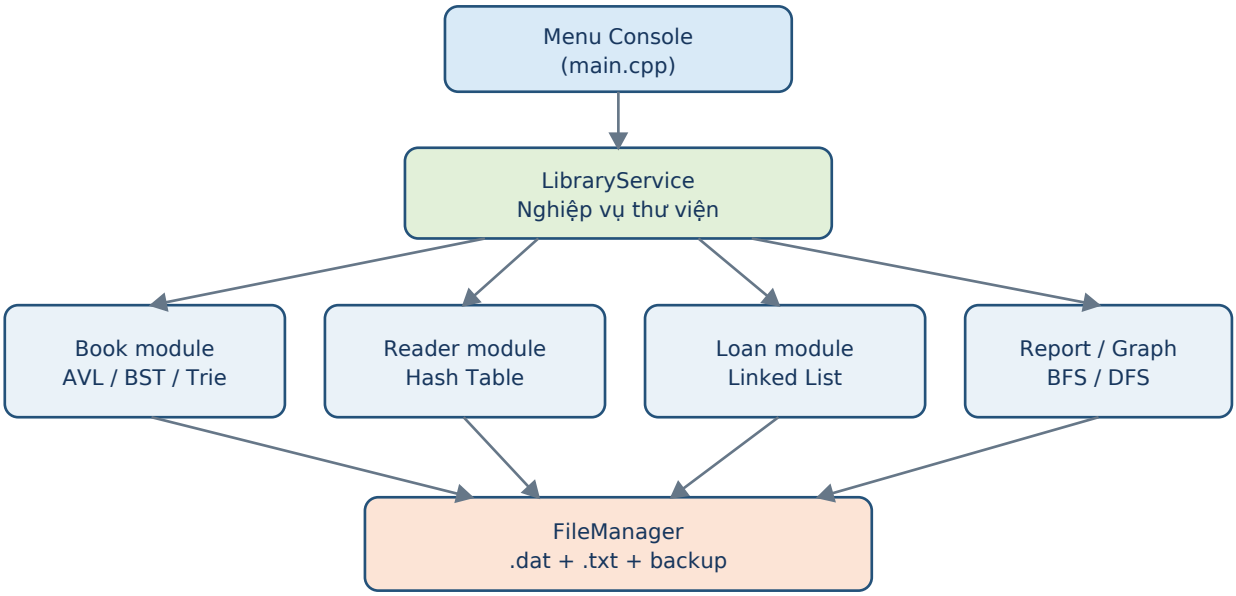
Nguyên tắc

1. Không dùng CSDL. 2. Không viết toàn bộ chương trình trong một file. 3. Mỗi cấu trúc dữ liệu phải gắn với chức năng được đề bài yêu cầu. 4. Tách dữ liệu nghiệp vụ khỏi giao diện menu. 5. Chỉ ghi file khi dữ liệu hợp lệ và kiểm tra lỗi đọc/ghi.

1. Kiến trúc module và thư mục

Thư mục/file	Trách nhiệm
include/Book.h	Khai báo struct Book và các hàm liên quan sách.
include/Reader.h	Khai báo struct Reader và các hàm liên quan độc giả.
include/Loan.h	Khai báo struct Loan và trạng thái mượn/trả.
include/AVLTree.h	Khai báo cây AVL quản lý sách theo ISBN.
include/BookNameBST.h	Khai báo BST để duyệt danh mục theo tên.
include/ReaderHashTable.h	Khai báo bảng băm độc giả, separate chaining.
include/LoanList.h	Khai báo danh sách liên kết đơn phiếu mượn.
include/TransactionArray.h	Khai báo mảng động lưu lịch sử giao dịch.
include/BookGraph.h	Khai báo đồ thị danh sách kề và BFS/DFS.
include/Trie.h	Khai báo Trie tự cài đặt cho autocomplete tên sách.
include/FileManager.h	Khai báo các hàm đọc/ghi .dat, log .txt và backup.
include/LibraryService.h	Điều phối nghiệp vụ: thêm sách, mượn/trả, kiểm tra ràng buộc.
src/*.cpp	Phần định nghĩa hàm tương ứng; main.cpp chỉ xử lý menu và gọi service.
data/	Dữ liệu mẫu và file .dat/.txt khi chạy.
docs/	SRS.pdf, Design.pdf, ảnh sơ đồ, hướng dẫn.
tests/	Test cases và chương trình test nếu có.

Cây module (mức khái quát)



Giải thích: menu chỉ nhận lựa chọn và hiển thị kết quả; LibraryService xử lý quy tắc nghiệp vụ; các module CTDL chịu trách nhiệm lưu/tìm/duyệt dữ liệu; FileManager phụ trách đọc ghi và sao lưu. Cách chia này tránh để logic nghiệp vụ bị trộn vào mã menu.

2. Thiết kế cấu trúc dữ liệu

2.1 Các struct nghiệp vụ

Các struct dưới đây là bản thiết kế dữ liệu đề xuất. Có thể điều chỉnh kiểu ngày và cách lưu chuỗi khi cài đặt, nhưng phải giữ đủ trường theo đề.

```
// Book.h
struct Book {
    string isbn;
    string title;
    string author;
    string category;
    int publicationYear;
    int totalQuantity;
    int availableQuantity;
};

// Reader.h
struct Reader {
    string cmnd;
    string fullName;
    string email;
    string phone;
    string cardType;
};

// Loan.h
struct Loan {
    string loanId;
    string readerCmnd;
    string isbn;
    string borrowDate;   // định dạng đề xuất: YYYY-MM-DD
    string dueDate;      // ngày hẹn trả
    string returnDate;   // rỗng khi chưa trả
    bool isReturned;
    int overdueDays;
    double fineAmount;
};

// TransactionArray.h
struct Transaction {
    string transactionId;
    string loanId;
    string transactionType; // BORROW hoặc RETURN
    string transactionDate;
    string note;
};
```

Vì sao có availableQuantity?

totalQuantity là tổng số bản thuộc thư viện; availableQuantity là số bản hiện có thể cho mượn. Khi lập phiếu mượn, availableQuantity giảm 1; khi trả sách, số này tăng 1. Cần kiểm tra không để số lượng khả dụng âm hoặc lớn hơn tổng số lượng. Nếu nhóm muốn tránh lưu trùng dữ liệu, có thể tính số bản khả dụng từ tổng số lượng và các phiếu đang mượn, nhưng khi đó phải thống nhất một cách duy nhất.

2.2 Quan hệ giữa các dữ liệu

- Một Reader có thể có nhiều Loan.
- Một Book có thể xuất hiện trong nhiều Loan theo thời gian.
- Mỗi Loan liên kết đúng một Reader và một Book theo readerCmnd và isbn.
- Transaction ghi lại sự kiện mượn/trả để phục vụ lịch sử và báo cáo.

2.3 Ánh xạ cấu trúc dữ liệu theo yêu cầu đề

CTDL	Khóa/nội dung	Thao tác chính	Độ phức tạp dự kiến
AVL	ISBN	Thêm, tìm, xóa sách	O(log n) tìm/thêm/xóa

CTDL	Khóa/nội dung	Thao tác chính	Độ phức tạp dự kiến
BST	Tên sách làm khóa	Duyệt in-order để in tên tăng dần	Duyệt O(n); thao tác phụ thuộc chiều cao h
Hash Table + chaining	CMND	Thêm/tìm/xóa độc giả	Trung bình O(1) nếu hàm băm/phân bố tốt; xấu nhất O(n)
Linked List	Loan nodes	Thêm phiếu, duyệt/tìm phiếu	Thêm đầu O(1); tìm O(n)
Dynamic Array	Transaction	Thêm giao dịch, duyệt lịch sử	Thêm cuối trung bình O(1) khấu hao; truy cập chỉ số O(1)
Graph adjacency list	Sách và cạnh liên quan	BFS/DFS gợi ý	O(V+E)
Trie	Ký tự trong tên sách	Thêm tên, tìm tiền tố	Tìm tiền tố O(L), chưa tính thời gian liệt kê kết quả; L là độ dài tiền tố

3. Quy tắc thiết kế quan trọng

3.1 Khóa và trùng dữ liệu

ISBN được dùng làm khóa của AVL, CMND là khóa của bảng băm. Khi thêm mới, phải tìm trước để từ chối khóa trùng. Với BST theo tên, nhiều sách có thể trùng tên; nên dùng khóa ghép (tên chuẩn hóa + ISBN) để không làm mất sách trùng tên. Quy tắc so sánh chuỗi tiếng Việt cần được thống nhất; bản đơn giản có thể so sánh theo chuỗi byte/chuỗi đã chuẩn hóa, nhưng thứ tự dấu tiếng Việt có thể chưa đúng thứ tự từ điển tiếng Việt.

3.2 Quy tắc mượn — trả

Luồng mượn đề xuất:
1. Tìm độc giả theo CMND; nếu không có thì dừng.
2. Tìm sách theo ISBN; nếu không có hoặc availableQuantity = 0 thì dừng.
3. Kiểm tra giới hạn số sách đang mượn của độc giả theo quy tắc được thống nhất.
4. Kiểm tra ngày mượn/ngày hẹn trả hợp lệ.
5. Tạo Loan có isReturned = false; giảm availableQuantity; ghi Transaction BORROW; lưu dữ liệu.

Luồng trả đề xuất:
1. Tìm Loan theo loanId; từ chối nếu không tồn tại hoặc đã trả.
2. Ghi returnDate; tính overdueDays = max(0, số ngày giữa returnDate và dueDate).
3. Tính fineAmount theo mức phạt/ngày đã được thống nhất; chuyển isReturned = true.
4. Tăng availableQuantity; ghi Transaction RETURN; lưu dữ liệu.

3.3 Quy tắc đồ thị gợi ý sách

Mỗi đỉnh đại diện một ISBN. Đề yêu cầu gợi ý sách cùng thể loại, nên có thể nối hai đỉnh khi hai sách cùng thể loại. Nếu nối mọi cặp cùng thể loại, đồ thị có thể có nhiều cạnh; cần tránh thêm cạnh trùng. BFS/DFS từ ISBN ban đầu sẽ tìm các sách có liên hệ. Nên loại chính sách đang chọn khỏi danh sách gợi ý và chỉ hiển thị các sách còn tồn tại.

3.4 Thiết kế Trie

Đề cho phép chọn một CTDL tự tìm hiểu; bản thiết kế chọn Trie vì khớp trực tiếp yêu cầu autocomplete. Mỗi nút chứa các nhánh ký tự và cờ đánh dấu kết thúc tên sách. Khi nhập tiền tố, đi theo các ký tự của tiền tố; nếu tìm được nút tương ứng thì duyệt các nhánh phía dưới để lấy các tên bắt đầu bằng tiền tố đó. Trie phải tự cài đặt, không thay bằng thư viện có sẵn.

4. Thiết kế menu

```
========== QUAN LY THU VIEN ==========
1. Quan ly sach
2. Quan ly doc gia
3. Muon - tra sach
4. Thong ke - bao cao
5. Goi y / tim kiem nang cao
6. Sao luu du lieu
0. Luu va thoat
Lua chon:
```

Menu con	Các lựa chọn dự kiến
Quản lý sách	Thêm / sửa / xóa / tìm ISBN / in theo ISBN / in theo tên / autocomplete / import / export / quay lại
Quản lý độc giả	Thêm / sửa / xóa / tìm CMND / thống kê theo loại thẻ / quay lại
Mượn–trả	Lập phiếu mượn / trả sách / tìm phiếu / xem phiếu đang mượn / quay lại
Thống kê	Lượt mượn theo tháng / thể loại / độc giả / Top 10 sách / Top 10 độc giả / xuất báo cáo / quay lại

Menu con	Các lựa chọn dự kiến
Gợi ý nâng cao	Gợi ý cùng thể loại bằng BFS/DFS / tìm tiền tố bằng Trie / gợi ý từ lịch sử mượn / quay lại

Luồng khởi động và thoát

Khởi động → khởi tạo cấu trúc dữ liệu → đọc file .dat nếu tồn tại → ghi log khởi động → hiện menu. Khi người dùng chọn chức năng, menu kiểm tra lựa chọn và gọi hàm nghiệp vụ tương ứng. Khi thoát, xác nhận lưu → ghi dữ liệu → ghi log → kết thúc. Nếu file chưa tồn tại ở lần chạy đầu, chương trình khởi tạo dữ liệu rỗng và thông báo rõ; không nên xem file thiếu là lỗi nghiêm trọng trong lần chạy đầu.

5. Thiết kế lưu trữ file

5.1 Các file dữ liệu đề xuất

Tên file	Nội dung
data/books.dat	Danh sách sách; đọc/ghi theo định dạng nhị phân đã thống nhất.
data/readers.dat	Danh sách độc giả.
data/loans.dat	Danh sách phiếu mượn.
data/transactions.dat	Lịch sử giao dịch/mảng động.
data/library.log.txt	Log sự kiện, lỗi đọc/ghi, mượn/trả.
data/backup/YYYY-MM-DD/	Bản sao dữ liệu theo ngày; cách đặt tên có thể điều chỉnh.

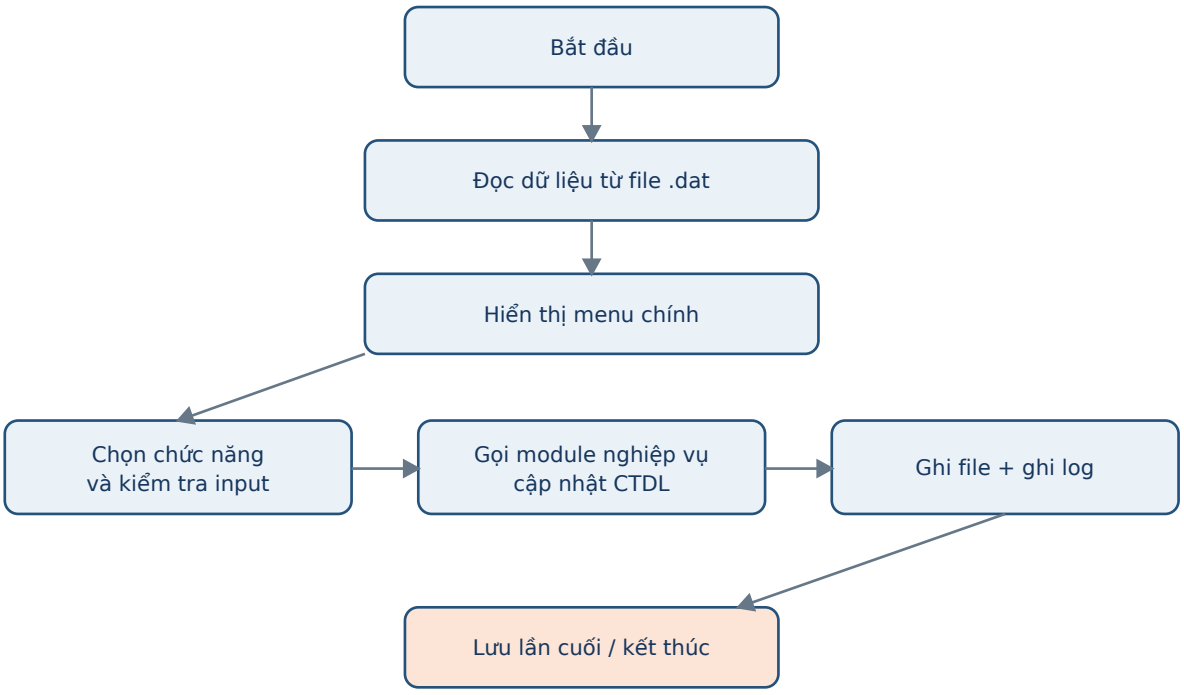
Lưu ý kỹ thuật

Không nên ghi trực tiếp toàn bộ struct có chứa std::string bằng cách dùng sizeof(struct) và write(reinterpret_cast(&obj;), sizeof(obj)); cách đó không lưu nội dung chuỗi an toàn vì std::string chứa con trỏ/quản lý bộ nhớ nội bộ. Với file nhị phân, hãy thiết kế hàm ghi/đọc từng trường (độ dài chuỗi + bytes, số nguyên, số thực) hoặc dùng định dạng tuần tự hóa riêng. Khi đọc phải kiểm tra số byte đọc được, giới hạn độ dài chuỗi và phiên bản định dạng.

5.2 Quy tắc an toàn dữ liệu

- Kiểm tra mở file thành công trước khi đọc/ghi.
- Không ghi đè file chính bằng dữ liệu chưa được kiểm tra; cân nhắc ghi file tạm rồi đổi tên.
- Nếu file bị thiếu, khởi tạo dữ liệu rỗng và ghi log.
- Nếu file hỏng, thông báo rõ và không âm thầm coi dữ liệu là hợp lệ.
- Backup theo ngày cần tránh ghi đè bản sao đã có nếu mục tiêu là giữ nhiều phiên bản.

6. Sơ đồ luồng tổng quát



Sơ đồ trên là luồng khái quát; khi cài đặt, sau khi hoàn thành một chức năng, chương trình quay về menu chính thay vì kết thúc. Nhánh thoát mới đi tới bước lưu lần cuối/kết thúc.

7. Kế hoạch kiểm thử sơ bộ

ID	Tình huống	Kết quả mong đợi
T01	Thêm sách với ISBN mới và dữ liệu hợp lệ	Thêm thành công; tìm lại thấy sách.
T02	Thêm sách trùng ISBN	Từ chối và thông báo ISBN đã tồn tại.
T03	Tìm ISBN không tồn tại	Thông báo không tìm thấy; chương trình tiếp tục chạy.
T04	Thêm độc giả trùng CMND	Từ chối dữ liệu trùng.
T05	Mượn khi sách có số lượng khả dụng bằng 0	Từ chối mượn, không tạo phiếu.
T06	Trả phiếu đã trả	Từ chối trả lần hai; không tăng số lượng thêm lần nữa.
T07	Khởi động khi file dữ liệu chưa tồn tại	Khởi tạo dữ liệu rỗng và thông báo/log.
T08	Không ghi được file	Báo lỗi rõ ràng; không báo lưu thành công giả.
T09	Autocomplete tiền tố có kết quả/không kết quả	Hiển thị các tên khớp hoặc thông báo không có kết quả.
T10	BFS/DFS từ một ISBN hợp lệ	Hiển thị các sách liên quan, không lặp sách trong kết quả.

8. Checklist Tuần 2 và việc cần xác nhận

Checklist Tuần 2 theo đề:
☐ Chọn và mô tả cấu trúc dữ liệu cho từng chức năng.
☐ Hoàn thiện sơ đồ module/lớp.
☐ Vẽ menu và luồng chương trình.
☐ Định nghĩa các struct dữ liệu.
☐ Đóng gói thành Design.pdf và đưa vào docs/.
☐ Commit thay đổi lên GitHub.

Câu hỏi cần chốt với giảng viên/nhóm trước khi lập trình

1. Mỗi độc giả được mượn tối đa bao nhiêu cuốn cùng lúc?
2. Mức phạt quá hạn là bao nhiêu tiền/ngày? Có miễn phạt hay không?
3. Ngày hẹn trả được nhập thủ công hay tự tính từ ngày mượn?
4. Có cho phép xóa sách/độc giả đã xuất hiện trong lịch sử không?
5. Định dạng ngày sẽ dùng YYYY-MM-DD hay định dạng khác?
6. Trie có được chọn làm CTDL tự tìm hiểu chính thức không? (Đề khuyến nghị Trie.)

9. Giải thích ngắn để trình bày với giảng viên

• Vì sao dùng AVL cho ISBN? Vì cần quản lý sách theo khóa và AVL tự cân bằng, tránh cây lệch quá cao.
• Vì sao dùng Hash Table cho CMND? Vì tìm độc giả theo khóa thường xuyên; hashing cho thời gian trung bình nhanh, separate chaining xử lý va chạm.
• Vì sao BST theo tên? Duyệt in-order có thể in theo thứ tự khóa tên; cần khóa phụ ISBN khi tên trùng.
• Vì sao Linked List cho phiếu mượn? Số phiếu tăng dần theo thời gian; danh sách cho phép thêm nút đơn giản, phù hợp yêu cầu bắt buộc.
• Vì sao Dynamic Array cho lịch sử? Truy cập theo chỉ số nhanh, dễ duyệt và tăng dung lượng khi cần.
• Vì sao Graph? Quan hệ cùng thể loại giữa sách có thể biểu diễn bằng cạnh; BFS/DFS tìm sách liên quan.
• Vì sao Trie? Tìm theo tiền tố là đúng bài toán của Trie; phải tự cài đặt theo yêu cầu đề.

10. Ghi chú giới hạn

Tài liệu thiết kế này là thiết kế khởi đầu cho tuần 2, chưa phải mã nguồn hoàn chỉnh. Các quy tắc chưa được đề quy định (mức phạt, giới hạn mượn, chuẩn hóa tiếng Việt) cần được xác nhận. Khi hiện thực, mọi cấu trúc bắt buộc phải thực sự tham gia xử lý dữ liệu và cần có test minh chứng.